Tema 2 · Anexo 1 · PKI con OpenSSL

SPD · Curso 2026/27 · Prof. Teodoro Rojo
Apuntes a partir de *SPD - 2A1. PKI con OpenSSL*. Este anexo es la base de la **Práctica 2 (PKI para HTTPS)**. "Nota" = aclaración añadida.

Índice

1. HTTPS y TLS
2. Certificados X.509
3. OpenSSL y creación de una PKI
4. HTTPS en Apache
5. SNI
6. Proxies HTTPS
7. Chuleta de comandos
8. Preguntas de repaso

---

1. HTTPS y TLS

1.1 Conceptos

- **HTTPS** (*HTTP over SSL*): extensión segura de HTTP, creada por **Netscape**. RFC 5246 y RFC 6176 (IETF).
- Usa la librería **SSL** (*Secure Socket Layer*) para un **transporte seguro**: **confidencialidad, autentificación e integridad**.
- **Puerto 443** (HTTP usa el 80). Mismos contenidos, pero sobre una conexión **TCP segura**.
- **TLS v1.0 = SSL 3.1.** Última versión: **TLS 1.3 (2018)**.

Nota: Hoy **SSL 2.0/3.0, TLS 1.0 y 1.1 están obsoletos**; solo se deben aceptar **TLS 1.2 y 1.3**.

Herramientas: capacidades TLS del navegador → howsmyssl.com · del servidor → ssllabs.com/ssltest.

1.2 Servicios de seguridad de TLS (importante)

Servicio	Cuándo	Cómo
Autentificación e intercambio de claves	Fase de establecimiento	Se autentica el **servidor** (y opcionalmente el **cliente**) con **certificados X.509** y **cifrado asimétrico** (RSA, ECDSA…)
Confidencialidad	Todo el intercambio (ambos sentidos)	**Cifrado simétrico** (AES, Camellia, ChaCha20, 3DES, RC4, IDEA…) + modo (GCM, CBC, CCM…)
Integridad	Todos los mensajes	**MAC / HMAC** (SHA-1, SHA-256/384…)

Nota: Es un **sistema híbrido** (ver tema 2): asimétrico para autenticar y acordar claves, simétrico para los datos.

1.3 Funcionamiento: dos subprotocolos

- **Handshake protocol:** establece la conexión, **autentica al servidor**, **negocia algoritmos** de cifrado e integridad y establece de forma segura una **clave pre-maestra** de la que se derivan las claves de sesión.
- **Record protocol: segmenta, cifra, controla la integridad** y (opcionalmente) **comprime** los datos sobre la sesión TCP.

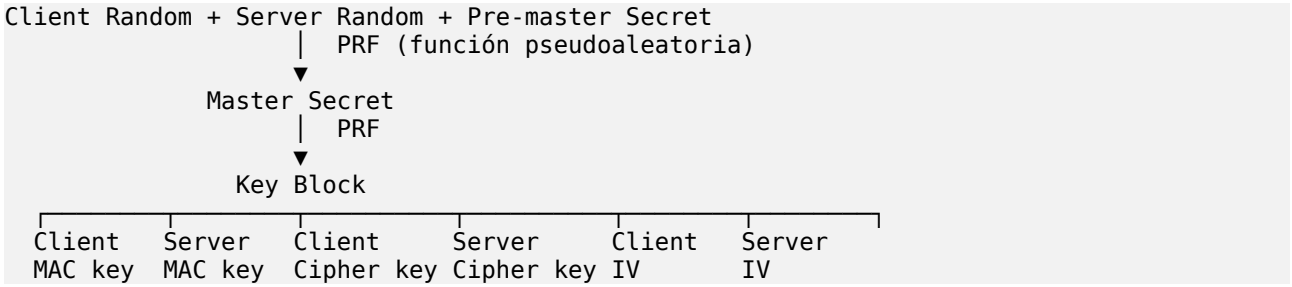
1.4 Handshake (importante)

1. El cliente envía **ClientHello** con sus preferencias (versiones, suites de cifrado, valor aleatorio).
2. El servidor responde **ServerHello** con lo que acepta + su **certificado** (identidad + clave pública).
3. El cliente **autentica al servidor** (verifica el certificado) y se establece la **clave pre-maestra**, de dos formas:

Modalidad	Establecimiento de clave	Autenticación
Handshake con RSA	RSA: el **cliente elige** la pre-maestra y la envía **cifrada con la clave pública** del certificado del servidor	RSA
Handshake con DH / DHE	La pre-maestra sale de un **intercambio Diffie-Hellman**	RSA / DSA / ECC

Nota: DHE (efímero) usa claves DH **distintas en cada sesión** → **PFS (*Perfect Forward Secrecy*)**: el robo futuro de la clave privada del servidor **no** permite descifrar sesiones pasadas. Con el handshake RSA **no hay PFS** (quien robe la clave privada puede descifrar la pre-maestra de sesiones grabadas). Por eso TLS 1.3 **eliminó** el intercambio RSA.

1.5 Obtención de las claves de sesión



→ **4 claves de sesión** (cifrado e integridad **para cada sentido**) + **IVs**.

1.6 TLS 1.2 vs TLS 1.3

	TLS 1.2	TLS 1.3
Suites de cifrado	Muchas (~20 recomendadas, cientos posibles)	**Solo 5**
Intercambio de claves	RSA, DH, DHE, ECDHE	**Solo (EC)DHE** → PFS siempre
Handshake	2-RTT	**1-RTT** (0-RTT en reconexión)
Cifrado	CBC, GCM, RC4…	**Solo AEAD** (AES-GCM, ChaCha20-Poly1305, AES-CCM)

Nota: Las 5 suites de TLS 1.3: `TLS_AES_128_GCM_SHA256`, `TLS_AES_256_GCM_SHA384`, `TLS_CHACHA20_POLY1305_SHA256`, `TLS_AES_128_CCM_SHA256`, `TLS_AES_128_CCM_8_SHA256`.

Nota: Cómo leer una suite TLS 1.2: `ECDHE-RSA-AES256-GCM-SHA384` = intercambio **ECDHE** · autenticación **RSA** · cifrado **AES-256** en modo **GCM** · hash **SHA-384** (para la PRF).

1.7 AES-GCM (*Galois/Counter Mode*)

- **Nonce** (*number used once*): distinto en cada sesión; se intercambia de forma segura al inicio (IV) y cambia en cada bloque gracias a un **contador**.
- **AAD** (*Additional Authenticated Data*): información **sin cifrar** (p. ej. cabeceras TCP/IP) que **sí** se usa para calcular la **authentication tag**.

1.8 HTTP/3 y QUIC

- **QUIC** (*Quick UDP Internet Connections*): protocolo de transporte de **Google**, estandarizado por el IETF. **HTTP/3** usa QUIC en lugar de TCP (**443/UDP**).
- Sobre **UDP** pero con confirmaciones, control de congestión y **TLS 1.3 integrado**.
- **1-RTT** para establecer conexión segura (0-RTT en reconexiones).

- **Persistencia** de la conexión aunque cambie la IP del cliente (p. ej. pasar de WiFi a 4G).
- **Multiplexación de streams** sin *Head-of-Line Blocking*: perder un paquete de un flujo no bloquea a los demás.

---

2. Certificados X.509

2.1 Certificados y PKI

- **Certificado:** documento electrónico que asocia de forma fiable una **entidad** con sus atributos, sobre todo su **clave pública**.
- Debe estar **firmado por una CA de confianza**. Para verificarlo hay que comprobar su firma con la **clave pública de la CA** (que está, a su vez, en otro certificado).
- Una CA es válida si su certificado está **en la configuración del cliente** (p. ej. el navegador) o si, **subiendo por la jerarquía**, se llega a una **raíz de confianza**.
- **PKI (*Public Key Infrastructure*)** = el modelo jerárquico + los procedimientos para **crear, publicar, verificar y revocar** certificados.

2.2 X.509

- Estándar más extendido. **1988**, ITU e ISO, como parte del directorio **X.500**. Versión actual **v3** (admite **extensiones**).
- **Campos:** versión (1, 2, 3) · **número de serie** · **algoritmo de firma** · **emisor (Issuer)** · **validez** (Not Before / Not After) · **titular (Subject)** identificado por su **DN** · **clave pública** del titular · **firma de la CA** · **extensiones** (uso de la clave, nombres alternativos…).

2.3 Nombre Distintivo (DN)

Tipo	Campos
Geográficos	**C** (*Country*), **ST/SP** (*State/Province*), **L** (*Locality*)
Organizativos	**O** (*Organization*), **OU** (*Organizational Unit*)
Personales	**CN** (*Common Name*): nombre del usuario, su email o **el nombre del servidor web**

2.4 Ejemplo real: www.bbva.es

```
Version: 3
Serial: 0AFFB69F59DCBAA10240EE505267DBAA
Signature Algorithm: sha256WithRSAEncryption
Issuer:  C=US, O=DigiCert Inc, CN=DigiCert EV RSA CA G2      ← CA intermedia
Validity: Jan 27 2026 → Jan 26 2027                          ← 1 año
Subject: ... O=Banco Bilbao Vizcaya Argentaria S.A., CN=www.bbva.es
Public Key: rsaEncryption 2048 bit, Exponent 65537
Extensions:
  Subject Alternative Name: DNS:www.bbva.es, DNS:bbva.es, ...   ← SAN
  Key Usage (critical): Digital Signature, Key Encipherment
  Extended Key Usage: TLS Web Server Authentication, TLS Web Client Authentication
  CRL Distribution Points: http://crl3.digicert.com/...            ← revocación
  Certificate Policies: ... CPS: https://www.digicert.com/CPS
```

Nota: Fíjate en: el emisor es una **CA intermedia** (no la raíz), el **exponente público 65537** (valor estándar de *e* en RSA), y el certificado es **EV** (incluye jurisdicción, nº de registro…).

2.5 Tipos de certificado

Tipo	Para qué
Wildcard (comodín)	Todos los **subdominios** de un dominio: `*.ceu.es` (www, mail, vpn…)
SAN (*Subject Alternative Names*)	**Varios dominios** en un certificado: `www.ceu.es` + `www.ceu.com`, con y sin `www`
EV (*Extended Validation*)	La CA ha **verificado la identidad** de la organización ("candado verde")

Nota: Let's Encrypt emite certificados **gratuitos** (validación de dominio, 90 días) de forma automática. **Atención: Los navegadores actuales solo aceptan certificados con SAN** (ignoran el CN para validar el nombre). Muy importante para la Práctica 2.

---

3. OpenSSL y creación de una PKI

3.1 Qué es OpenSSL

Librería criptográfica **open source** que ofrece:

- **Cifrado simétrico** (aes, des, 3des… con modos cbc, cfb…).
- **Hashes** (sha1, sha256, sha384…).
- **Cifrado asimétrico**: generar claves, cifrar, **firmar**, **generar certificados** y **gestionar PKIs** (RSA, DSA, ECDSA…).
- Última revisión según la diapositiva: **OpenSSL 4.0.2 (agosto 2026)**.

3.2 Comandos básicos

Para…	Comando
Ver versión y directorio de configuración (`OPENSSLDIR`)	`openssl version -a`
Listar algoritmos	`openssl list -cipher-algorithms` / `-digest-algorithms` / `-public-key-algorithms`
Valor aleatorio de 16 bytes (128 bits)	`openssl rand -hex 16`
Hash de un fichero	`openssl dgst -sha256 /etc/passwd`
Huellas de todo /etc	`find /etc -type f | xargs openssl dgst -sha1 > huellas.txt`
Cifrar / descifrar simétrico	`openssl enc -e -aes-128-cbc -in ejem.txt -out ejem.enc` / `openssl enc -d -aes-128-cbc -in ejem.enc -out ejem.txt2`
Generar clave RSA (protegida con 3DES)	`openssl genrsa -des3 -out servidor.key 2048`
Ver una clave	`openssl rsa -noout -text -in servidor.key`
Petición de certificado (CSR)	`openssl req -new -key servidor.key -out servidor.csr`
Firmar CSR con la CA	`openssl ca -in servidor.csr -out servidor.crt -days 730`
Exportar cert + clave a PKCS#12	`openssl pkcs12 -export -in juan.crt -inkey juan.key -out juan.p12`

Configuración: `openssl.cnf` (enlace simbólico) en **RedHat → `/etc/pki/tls`**, **Debian/Ubuntu → `/etc/ssl`**.

Nota: El caso de `find … | xargs openssl dgst` es un control **detectivo** del tema 1: guardas las huellas y, más adelante, si alguna cambia, sabes que el fichero fue modificado (idea de herramientas como *Tripwire/AIDE*).

3.3 Estructura de directorios de la CA (importante)

El `openssl.cnf` trae una CA de ejemplo (`CA_default`) que necesita:

Elemento	Contenido
`certs/`	Certificados **reconocidos**
`crl/`	Certificados **revocados**
`newcerts/`	Certificados **emitidos** (copia por n° de serie)

Elemento	Contenido
`private/`	**Clave privada de la CA** (`cakey.pem`)
`cacert.pem`	**Certificado de la CA**
`serial`	N° de serie del **siguiente** certificado
`crlnumber`	N° de secuencia de la siguiente CRL
`index.txt`	**Base de datos** (listado) de certificados emitidos

```
mkdir -p /etc/pki/tls && cd /etc/pki/tls
mkdir certs crl newcerts private
touch index.txt
echo 01 > serial
echo 01 > crlnumber
```

Atención: Sincronizar fecha y hora del equipo (si no, los certificados pueden aparecer "aún no válidos"). Haz **copia de seguridad de `openssl.cnf`** antes de editarlo.

3.4 El fichero openssl.cnf

Sección	Para qué
`[ ca ]`	Nombre de la CA por defecto
`[ <nombre_CA> ]`	Directorios y ficheros, valores por defecto (validez...) y **policy**. **Atención:** Comprobar que **`dir`** apunta a la raíz de la PKI
`[ <policy> ]`	Para cada campo del DN: **`match`** (debe coincidir con la CA), **`supplied`** (obligatorio) u **`optional`**
`[ req_distinguished_name ]`	Valores por defecto que se proponen para el DN

Extensiones (definen el *uso* del certificado):

```
[ v3_ca ]                       # certificado raíz de la CA
basicConstraints = critical, CA:TRUE, pathlen:3
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid:always,issuer
keyUsage = critical, cRLSign, keyCertSign

[ server ]                      # servidor (obsoleto sin SAN)
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth

[ user ]                        # usuario / cliente
basicConstraints = CA:FALSE
keyUsage = digitalSignature
extendedKeyUsage = clientAuth

[ server_SAN ]                  # servidor con nombres alternativos ← EL QUE HAY QUE USAR
basicConstraints = CA:FALSE
keyUsage = nonRepudiation, digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName = @alt_names

[ alt_names ]
DNS.1 = www.efirel.com
DNS.2 = efirel.com
```

Nota: Claves: `CA:TRUE` solo en la CA (puede firmar otros certificados); `pathlen:3` = máximo de CAs intermedias por debajo; `keyCertSign` y `cRLSign` = puede firmar certificados y CRLs; `serverAuth`/`clientAuth` = solo vale para servidor / cliente. Más info: `man x509v3_config`.

3.5 Crear la CA (paso a paso) (importante)

1. Crear la **estructura** de directorios y ficheros.
2. Editar **openssl.cnf**: DN por defecto y nuevos contextos (`server_SAN`, `user`...).
3. **Claves RSA de la CA + certificado autofirmado** (extensión `v3_ca`):

```
openssl genrsa -aes256 -out private/cakey.pem 2048
openssl req -new -key private/cakey.pem -out ca-csr.pem
openssl req -x509 -extensions v3_ca -in ca-csr.pem -out cacert.pem \
            -key private/cakey.pem -days 3652
```

4. **CRL** inicial (vacía):

```
openssl ca -gencrl -out crl.pem
```

3.6 Emitir certificados (importante)

Servidor HTTPS (CN = www.efirel.com, extensión `server_SAN`):

```
openssl genrsa -out efirel-key.pem 2048
openssl req -new -key efirel-key.pem -out efirel-csr.pem
openssl ca -extensions server_SAN -in efirel-csr.pem -out efirel-crt.pem -days 730
```

Después, **instalar `cacert.pem` como raíz de confianza** en los navegadores de los clientes.

- En Windows, en el almacén del **USUARIO**, no en el del equipo.
- Si la clave del servidor tiene **frase de paso**, Apache la pedirá **cada vez que arranque**.

Clientes (extensión `user`) + exportación **PKCS#12** (para importar en el navegador):

```
openssl req -new -keyout juan-key.pem -out juan-csr.pem -noenc
openssl ca -extensions user -in juan-csr.pem -out juan-crt.pem -days 730
openssl pkcs12 -export -in juan-crt.pem -inkey juan-key.pem -out juan-crt.p12
```

Nota: Flujo general de cualquier certificado: `clave privada → CSR (petición) → la CA firma → certificado`.

Revocar un certificado y regenerar la CRL:

```
openssl ca -revoke juan-crt.pem
openssl ca -gencrl -out crl.pem
```

---

4. HTTPS en Apache

- Módulo **`mod_ssl`**. Se configura como **servidor virtual en el puerto 443** (en `httpd.conf` o en un fichero incluido: `Include conf.d/*.conf`).

Directiva	Función
`Listen`	Puerto
`DocumentRoot`	Raíz de contenidos
`SSLEngine on`	Activa SSL/TLS
`SSLCertificateFile`	**Certificado** del servidor
`SSLCertificateKeyFile`	**Clave privada** del servidor
`SSLCACertificateFile`	Certificados de las **CA reconocidas** (para validar clientes)
`SSLVerifyClient require`	**Exige certificado de cliente**
`SSLVerifyDepth`	Profundidad máxima de la cadena
`SSLCARevocationFile` / `SSLCARevocationCheck chain`	**CRL** y comprobación de revocación
`SSLRequire`	Restringe el acceso según campos del certificado

```
Listen 0.0.0.0:443
<VirtualHost *:443>
    SSLEngine on
    SSLCertificateFile     /etc/pki/tls/certs/servidor.crt
    SSLCertificateKeyFile  /etc/pki/tls/private/servidor.key
    SSLVerifyClient require
    SSLVerifyDepth 10
    SSLCACertificateFile   /etc/pki/tls/cacert.pem
```

```
    SSLCARevocationFile    /etc/pki/tls/certs/crl.pem
    SSLCARevocationCheck chain
</VirtualHost>
```

Restringir una zona a un usuario concreto:

```
<Location /admin/>
    SSLRequire ( %{SSL_CLIENT_S_DN_O} eq "EPS CEU" && %{SSL_CLIENT_S_DN_CN} eq "Juan Lopez" )
</Location>
```

Atención: TLS 1.3 no permite autenticar al cliente después del handshake (*Post-Handshake Authentication*). Si pones `SSLVerifyClient require` dentro de un `<Location>`/`<Directory>` da el error **"Cannot perform Post-Handshake Authentication"**. Solución: poner la directiva **en la raíz del servidor** o **desactivar TLS 1.3** (`SSLProtocol all -SSLv3 -TLSv1.3`).

Securizar HTTPS (configuración recomendada)

```
SSLProtocol          all -SSLv3 -TLSv1 -TLSv1.1
SSLCipherSuite       ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-
CHACHA20-POLY1305:...
SSLHonorCipherOrder  on
SSLCompression       off
SSLSessionTickets    off
```

Directiva	Por qué
`SSLProtocol`	Solo **TLS 1.2 / 1.3**
`SSLCipherSuite`	Solo suites **ECDHE + AEAD** (seguras, con PFS)
`SSLHonorCipherOrder on`	**Manda el orden del servidor**, no el del cliente
`SSLCompression off`	Evita ataques tipo **CRIME**
`SSLSessionTickets off`	Garantiza **PFS** (el descubrimiento de una clave no compromete sesiones anteriores)

5. SNI (*Server Name Indication*)

- **Problema:** el handshake TLS ocurre **antes** de las cabeceras HTTP, así que el servidor **no sabía a qué dominio** iba la petición al enviar su certificado → **un solo sitio HTTPS por IP** (caro).
- **SNI** (extensión TLS, **2003**): el cliente indica el **nombre de dominio en el ClientHello**. Soportado en navegadores y servidores desde **2010-2012**.
- En Apache: **servidores virtuales por nombre** con `ServerName` y `ServerAlias`.

---

6. Proxies HTTPS

Proxy (pasarela): servicio que hace de **intermediario** entre clientes y servidores, normalmente en la frontera red privada ↔ Internet. Funciones: control de acceso, filtrado de destinos, análisis de contenidos, **caché**, anonimato, cifrado…

	Forward proxy (proxy directo)	Reverse proxy (proxy inverso)
Controla	Tráfico **saliente** de los clientes hacia Internet	Peticiones **entrantes** del exterior hacia nuestros servicios
Seguridad	Limita peticiones y analiza respuestas	Filtra peticiones, analiza URLs, **descifra SSL**, "limpia" respuestas
Eficiencia	Caché de contenidos recibidos	Caché de contenidos servidos
Otros	Explícito o transparente	Identificación de destino, reescritura de URL, **balanceo**, escalabilidad

HTTPS a través de un forward proxy:

- **Explícito:** el cliente usa el método **`CONNECT`** (no GET) y el proxy solo hace de túnel.
- **Inspección TLS / transparente (*SSL Bumping*):** el proxy **intercepta el handshake** (doble conexión), **genera "al vuelo" un certificado** para el dominio destino firmado por **la CA del proxy**, que **el cliente debe tener instalada como de confianza**.
- **Atención:** Normalmente **no** se activa con datos o servicios privados (**RGPD**).

Nota: Es, literalmente, un **ataque MitM consentido**: por eso exige instalar la CA del proxy en el cliente.

Ejercicio de clase: mitmproxy

1. Arrancar `mitmproxy --listen-host 0.0.0.0 --listen-port 8080`.
2. Configurar el SO/navegador para usar el proxy → la primera web HTTPS da **error de certificado**.
3. Ir a **http://mitm.it**, descargar e instalar la **CA raíz de mitmproxy** → ya se navega por HTTPS y el proxy lo **registra todo**.
4. Enviar el formulario de `https://httpbin.org/forms/post` con datos ficticios y localizar en el proxy el **POST a /post** con los datos en claro.
5. Salir (q), **quitar el proxy** y **eliminar el certificado raíz** de mitmproxy.

Ejercicio de clase: analizar www.ceu.es

Con ssllabs.com o el navegador (F12 → *Security*): cadena de confianza (intermedia y raíz), CN, validez, SAN, tipo y longitud de clave, algoritmo de firma; versiones TLS y suites del servidor; algoritmo para dispositivos **sin AES-NI** (**Nota: ChaCha20-Poly1305**); disponibilidad de **PFS**; protección frente a **contenido mixto**.

---

7. Chuleta de comandos OpenSSL

```
# --- Información ---
openssl version -a
openssl x509 -in cert.pem -noout -text          # ver un certificado
openssl req  -in peticion.csr -noout -text      # ver una CSR
openssl crl  -in crl.pem -noout -text           # ver una CRL
openssl verify -CAfile cacert.pem cert.pem      # verificar cadena
openssl s_client -connect www.ceu.es:443 -servername www.ceu.es   # probar un servidor TLS

# --- CA ---
openssl genrsa -aes256 -out private/cakey.pem 2048
openssl req -new -key private/cakey.pem -out ca-csr.pem
openssl req -x509 -extensions v3_ca -in ca-csr.pem -key private/cakey.pem -out cacert.pem -
days 3652
openssl ca -gencrl -out crl.pem

# --- Servidor ---
openssl genrsa -out srv-key.pem 2048
openssl req -new -key srv-key.pem -out srv-csr.pem
openssl ca -extensions server_SAN -in srv-csr.pem -out srv-crt.pem -days 730

# --- Cliente ---
openssl req -new -keyout user-key.pem -out user-csr.pem -noenc
openssl ca -extensions user -in user-csr.pem -out user-crt.pem -days 730
openssl pkcs12 -export -in user-crt.pem -inkey user-key.pem -out user.p12

# --- Revocación ---
openssl ca -revoke user-crt.pem
openssl ca -gencrl -out crl.pem
```

(**Nota:** `x509 -text`, `verify` y `s_client` no están en las diapositivas pero son muy útiles para **comprobar** y sacar capturas en la práctica.)

---

8. Preguntas de repaso

1. ¿Qué servicios de seguridad ofrece TLS y con qué tipo de criptografía se consigue cada uno?
2. Diferencia entre *handshake protocol* y *record protocol*.

3. Handshake con RSA vs con DHE. ¿Cuál ofrece PFS y por qué?
4. ¿Cuántas claves de sesión se derivan del *key block* y para qué sirve cada una?
5. Tres diferencias entre TLS 1.2 y TLS 1.3.
6. Interpreta la suite `ECDHE-RSA-AES128-GCM-SHA256`.
7. Campos de un certificado X.509 y componentes del DN.
8. Wildcard vs SAN vs EV. ¿Por qué hoy es obligatorio usar SAN?
9. Directorios y ficheros de una CA en OpenSSL. ¿Para qué sirven `serial` e `index.txt`?
10. ¿Qué significan `CA:TRUE`, `pathlen`, `keyCertSign`, `serverAuth` y `clientAuth`?
11. Pasos (y comandos) para crear una CA, emitir un certificado de servidor y uno de cliente.
12. ¿Cómo se revoca un certificado y cómo se entera Apache?
13. ¿Por qué da error `SSLVerifyClient require` dentro de un `<Location>` con TLS 1.3?
14. ¿Qué problema resuelve SNI?
15. Forward vs reverse proxy. ¿Cómo puede un proxy inspeccionar tráfico HTTPS y qué implica para el cliente?